

# GOLANG FOR DEVOPS



**DAY-1**

# INTRODUCTION TO GOLANG AND DEVOPS



# Overview of GoLang

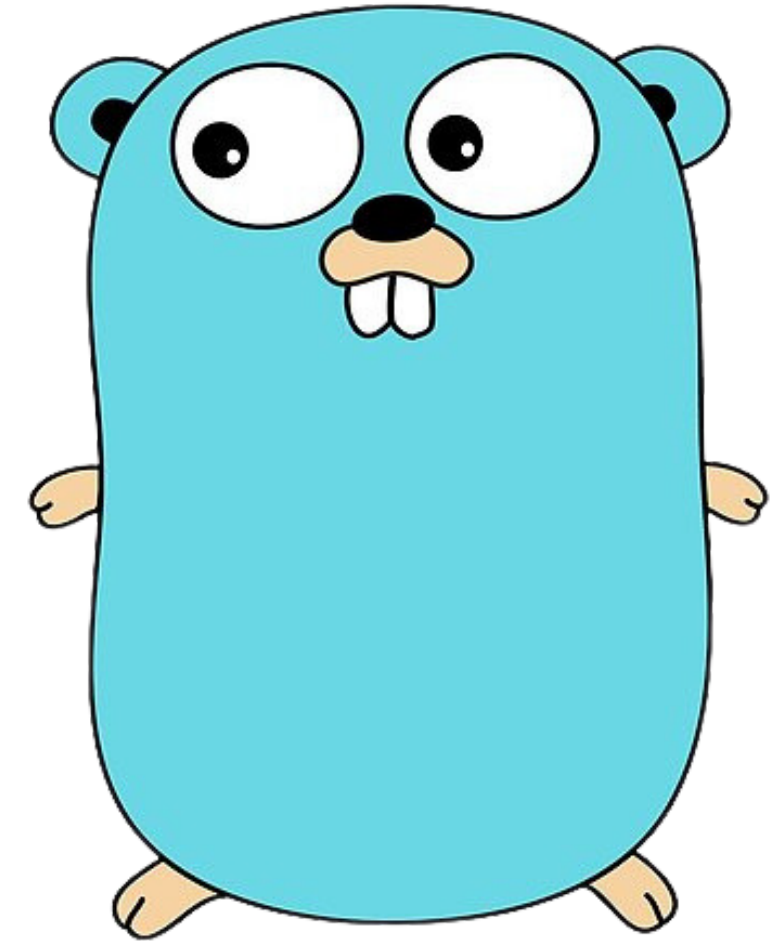


@Sandip Das



# What is GoLang?

GoLang, also known as Go, is an open-source programming language developed by Google. It was created to address the issues of software development at Google, and it has since gained popularity in the broader software industry.



@Sandip Das

# Go History

- 2007: Development of Go begins at Google, initiated by Robert Griesemer, Rob Pike, and Ken Thompson.
- 2009: Go is publicly announced in November, and version 1.0 is released as an open-source programming language.
- 2012: Go version 1 is officially released in March, providing a stable foundation for the language.
- 2015: Go 1.5 is released in August, featuring a complete rewrite of the compiler, moving from C to Go.
- 2016: The Go 1.7 release in August includes a new compiler back end for 64-bit x86 systems, based on static single-assignment form, improving performance.
- 2017: Go 1.8 is released in February, continuing improvements in performance, garbage collection, and the standard library.
- 2018: Go 1.11, released in August, introduces Go modules, a new dependency management system that makes dependency version information explicit and easier to manage.
- 2019: Go 1.13, released in September, includes improvements to Go modules and the introduction of error wrapping in the standard library.
- 2020: Go 1.15, released in August, focuses on improvements to the toolchain, runtime, and libraries, with a significant change being the reduction of the typical binary size.
- 2021: Go 1.17, released in August, brings changes to the language and improvements to the core library and toolchain.
- 2022: Go 1.18, introduces generics along with bug fixes and security fixes for compiler.
- 2023: Go 1.21 overall language improvements

Go version 1.0 publicly announced as open source programming language

2009

Go 1.5 is released with a complete rewrite of the compiler

2015

Go 1.8 was released in February with improvements in performance, garbage collection, and the standard lib

2017

Go 1.13, released, with improvements in go modules and error handling

2019

Go 1.17, released with core library and tool chain improvements

2021

Development of Go begins at Google

2007

Go Stable version 1.x is officially released

2012

The Go 1.7 release with a new compiler back end for 64-bit x86 systems

2016

Go 1.11 released , introduces Go modules

2018

Go 1.15, released with improvements in toolchain, runtime, reduced binary size

2020

Source: Official Go release page



@Sandip Das

# Features of GoLang

## 01 Simplicity and Readability

Go is designed to be simple and easy to understand. Its syntax is clean and concise, which makes it easy to read and write.

## 02 Concurrency Support

One of the most notable features of Go is its built-in support for concurrency. Go routines (lightweight threads) and channels (for communication between Go routines) make concurrent programming more accessible and efficient.

## 03 Fast Compilation

Go compiles very quickly, which enhances the productivity of developers. This is partly due to its simplicity and the way it handles dependencies.

## 04 Static Typing

Go is statically typed, which means that types are checked at compile time. This can lead to more reliable and efficient code.



# Features of GoLang

## 05 Garbage Collection

It has a powerful garbage collector that helps manage memory automatically, reducing the risk of memory leaks and other memory-related errors.

## 06 Standard Library

Go comes with a rich standard library that covers a wide range of functionalities, from handling I/O to networking.

## 07 Cross-Platform

Go supports cross-platform development, making it possible to build software that runs on various operating systems, including Windows, Linux, and macOS.

## 08 Tools and Ecosystem

Go has a growing ecosystem with a variety of tools for testing, debugging, and dependency management.



# Usecases

## 01 Web Development

Go is often used for building web servers, RESTful APIs, and microservices due to its efficiency and scalability.

## 02 Cloud and Network Services

Its performance and concurrency features make it suitable for cloud computing, networking services, and distributed systems.

## 03 DevOps and Tooling

Go is popular in the DevOps community for building tools and utilities, thanks to its simplicity and ability to produce statically linked binaries that are easy to deploy. e.g. Docker, Kubernetes, Prometheus, Terraform, Etcd, Grafana, Hugo , InfluxDB, CockroachDB etc

## 04 Data Processing

Go's performance characteristics make it suitable for data processing tasks that require concurrency and efficiency.





# It's time to Install Go

## 01 Follow Official Installation Instruction

Go to: <https://go.dev/doc/install> then follow simple instructions

## 02 Download Go

download the Go installer from the official Go website: [golang.org](https://golang.org). Choose the appropriate version for your operating system. After downloading follow instructions as per step 1





# Getting Started...

## 01 Make Sure You have good An integrated development environment (IDE)!

Good IDE makes life easier, if you are not sure which to choose, here are the most popular for Go:

LiteIDE , Visual Studio Code (VS Code), GoLand, Sublime Text

My personal all-time favorite is VS Code, which you can download from here: <https://code.visualstudio.com>

## 02 IDE installed ? let's run our sweet old first program!

```
package main
```

```
import "fmt"
```

```
func main() {  
    fmt.Println("Hello, World!")  
}
```



@Sandip Das

# Basic GoLang syntax & types





# Basic Syntax

## Packages

Every Go file starts with a package declaration, which provides a way for code to be reused. The main package is the starting point of a Go program.

```
package main
```

## Imports

This is where you include code from other packages. The `fmt` package, for example, is commonly used for formatted I/O.

```
import "fmt"
```

## Functions

Functions are declared with the `func` keyword. The main function is the entry point of a Go program.

```
func main() {  
    fmt.Println("Hello, World!")  
}
```



# Basic Syntax

## Variables

Variables can be declared using the var keyword, but Go also allows for short variable declaration with :=.

```
var x int = 10
y := 20 // Short declaration
```

## Control Structures

Go has control structures like if, for, switch, but no while or do while.

```
for i := 0; i < 10; i++ {
    fmt.Println(i)
}
```

## Error Handling

Go handles errors by returning an error type as a part of the function's return value.

```
func sqrt(x float64) (float64, error) {
    if x < 0 {
        return 0, errors.New("negative value")
    }
    return math.Sqrt(x), nil
}
```



# Types

## Basic Types

Includes integers (int, uint, int8, uint8, int32, uint32, int64, uint64), floating points (float32, float64), complex numbers (complex64, complex128), boolean (bool), and string (string).

## Composite Types

### Arrays:

Fixed-size list of elements of the same type.

```
var a [5]int
```

### Slices:

Dynamically sized, flexible view into the elements of an array.

```
s := []int{1, 2, 3}
```

### Structs:

Collection of fields.

```
type Person struct {  
    Name string  
    Age  int  
}
```

### Maps:

Collection of key-value pairs.

```
m := map[string]int{"foo": 1, "bar": 2}
```

# Types

## Pointers

Go has pointers, allowing you to pass references to values and records within your program.

```
var p *int
```

## Interfaces

An interface type is defined by a set of methods. A value of interface type can hold any value that implements those methods.

```
type Reader interface {  
    Read(p []byte) (n int, err error)  
}
```

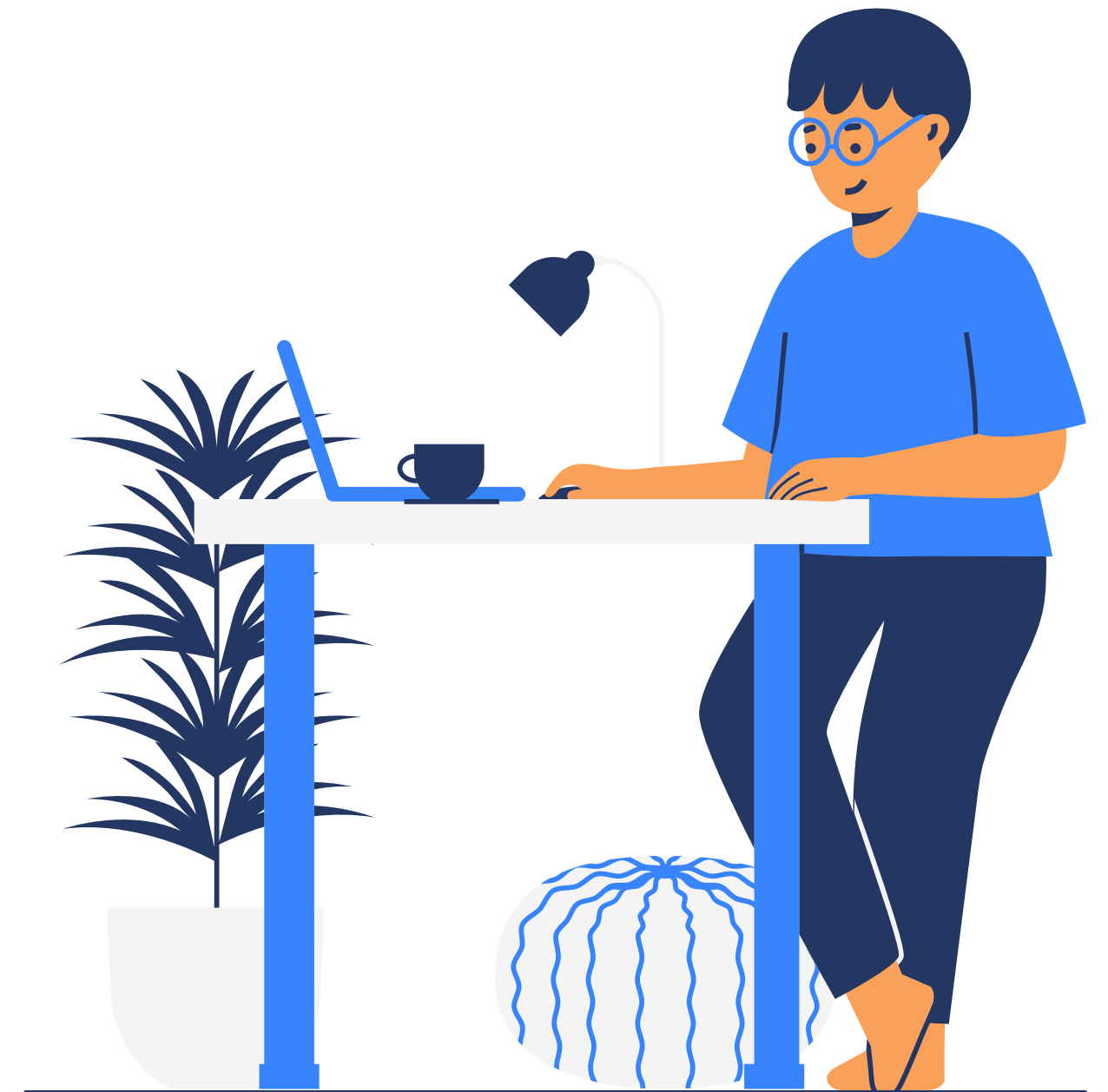
## Functions as Types

Functions can also be used as types, allowing for higher-order functions.

```
func compute(fn func(float64, float64) float64) float64 {  
    return fn(3, 4)  
}
```



# Introduction to DevOps and its principles

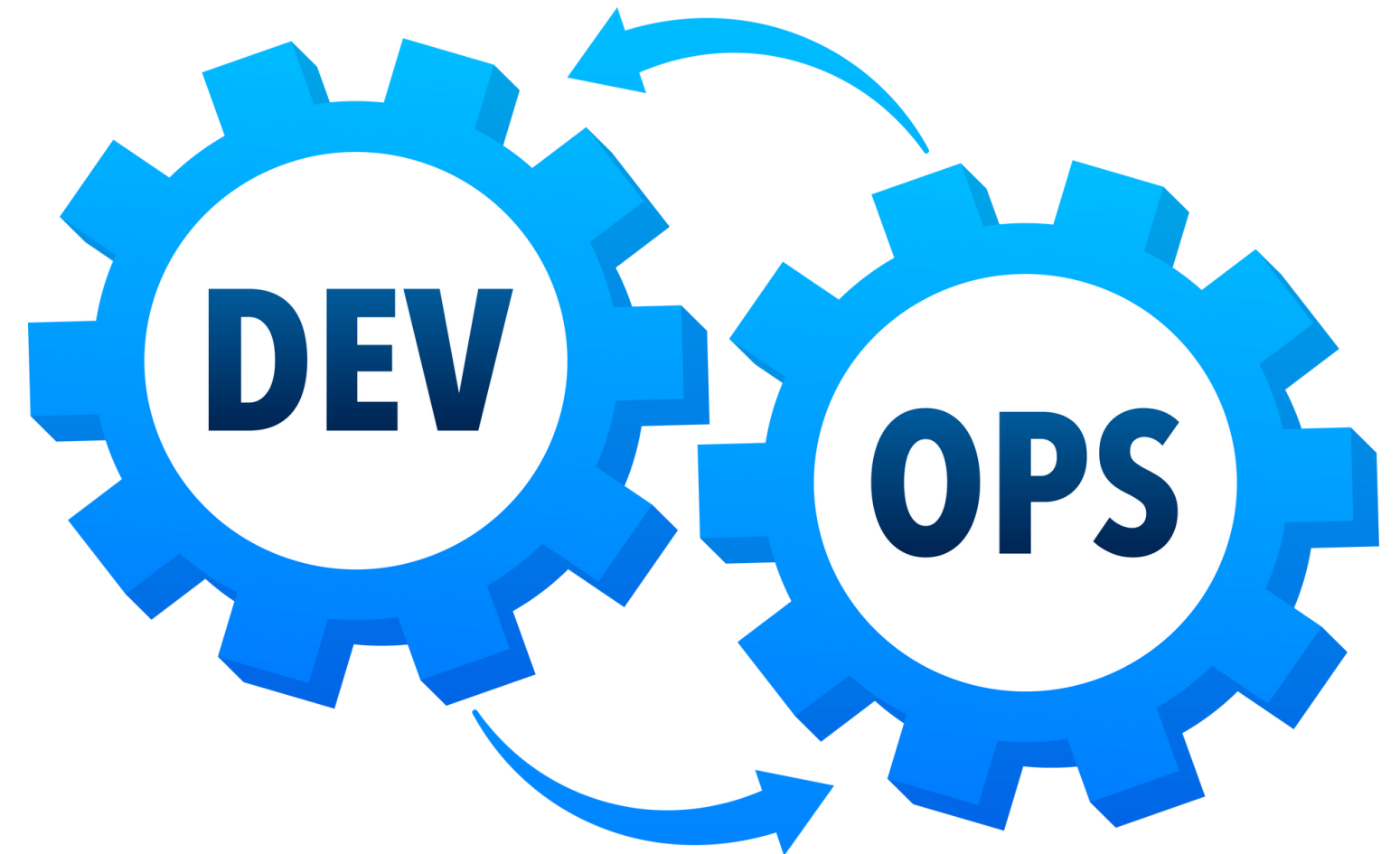


# What is DevOps?

DevOps is a set of practices and philosophies that aims to unify software development (Dev) and software operation (Ops).

The primary goal of DevOps is to shorten the systems development life cycle while delivering features, fixes, and updates frequently in close alignment with business objectives.

It emphasizes a high degree of collaboration between teams that were traditionally separate, and it often involves automating processes that historically have been manual and slow.



# Key Principles of DevOps



@Sandip Das

## Collaboration and Culture:

DevOps stresses a culture of collaboration where developers and operations teams work closely together. This approach breaks down silos and fosters a shared responsibility for the product's success

## Automation:

Automation is a cornerstone of the DevOps philosophy. This includes automating the integration, testing, deployment, and monitoring of software. Automation helps to increase efficiency, reduce errors, and speed up the delivery of software.

## **Continuous Integration and Continuous Delivery (CI/CD):**

CI/CD is a method to frequently deliver apps to customers by introducing automation into the stages of app development. The main concepts attributed to CI/CD are continuous integration, continuous delivery, and continuous deployment.

- **Continuous Integration** is a coding philosophy and set of practices that drive development teams to implement small changes and check in code to version control repositories frequently.
- **Continuous Delivery** is the approach of developing software in such a way that it can be released to production at any time.
- **Continuous Deployment** goes one step further than continuous delivery. Every change that passes all stages of your production pipeline is released to your customers. There's no human intervention, and only a failed test will prevent a new change to be deployed to production.

## **Monitoring and Feedback:**

Continuous monitoring of the system's performance is crucial. Feedback from monitoring systems is used to make adjustments and improvements.

## **Microservices Architecture:**

Many DevOps teams favor a microservices architecture, which structures an application as a collection of loosely coupled services. This approach enables small, independent teams to develop, deploy and scale their respective services independently.

## **Infrastructure as Code (IaC):**

IaC is the process of managing and provisioning computer data centers through machine-readable definition files, rather than physical hardware configuration or interactive configuration tools.

## **Lean and Agile:**

DevOps incorporates aspects of the Lean and Agile methodologies. It focuses on streamlining workflows, reducing waste, and increasing efficiency.

## **Resilience and Security:**

Emphasizing resilience and security as integral parts of the software lifecycle. This includes adopting security practices throughout the development process and building systems that are resilient to failure.



# How GoLang fits into the DevOps ecosystem

1. **Custom Infrastructure Automation:** GoLang is used to create custom plugins and extensions for IaC tools like Terraform and Ansible.
2. **Kubernetes Development:** GoLang is ideal for building Kubernetes operators and custom controllers.
3. **Microservices Architecture:** GoLang's efficiency makes it a great choice for building microservices.
4. **CI/CD Pipeline Customization:** DevOps teams use GoLang to build custom CI/CD pipeline scripts and integrations.
5. **Monitoring and Observability:** GoLang is used for building custom monitoring agents and log aggregators.
6. **Cloud-Native Components:** GoLang is suited for building serverless functions and cloud-native services.
7. **Automation Tooling:** GoLang is used to create various automation scripts and utilities.
8. **Security Enhancements:** GoLang's memory safety features improve security in DevOps-related software.
9. **Performance and Scalability:** GoLang's concurrency support is crucial for highly concurrent systems.
10. **Rich Library Ecosystem:** GoLang benefits from a wide range of open-source libraries for DevOps tasks.



*Thank  
You*



[@LearnTechWithSandip](#)



**SUBSCRIBE**



**For staying till the end**

Follow on:  [Sandip Das](#)